# Rapport final — InclusiVenue

Date : 4 novembre 2026 · Version 2.0.0 · Auteur : Mohamed Haouari (avec assistance IA, cf. messages de commit).

## 1. Ce qui a été réalisé

**Backend** (Node 22, Express 4, Mongoose 8, Zod) — architecture routes → controllers → services → repositories, 49 chemins / 58 opérations REST documentés dans Swagger : - authentification JWT (cookie httpOnly SameSite=Strict ou Bearer), bcrypt, inscription participant/organisateur (jamais admin), connexion, déconnexion, profil, changement et réinitialisation du mot de passe, suspension immédiate ; - RBAC sur chaque route + règles de propriété dans les services ; rate limiting, helmet, CORS strict, anti-injection NoSQL, en-tête CSRF, erreurs centralisées sans fuite ; - événements : CRUD, 12 services d'accessibilité, géolocalisation, upload d'image vérifié et converti en WebP, modération (brouillon / en attente / publié / refusé avec motif / archivé), suppression encadrée ; - recherche plein texte + filtres (catégorie, ville, dates, gratuit/payant, services, places disponibles), 6 tris, pagination ; - réservation atomique sans surréservation, anti-doublon par index unique partiel, annulation avec délai, pointage, historique, export iCal ; - notifications in-app et e-mail (SMTP réel ou boîte de développement explicite), rappels J-1 idempotents ; - statistiques organisateur et administrateur calculées par agrégations ; paramètres plateforme, catégories, messages de contact ; seed de démonstration.

**Frontend** (React 18, Vite, React Router, React Hook Form + Zod) — 29 écrans en français : 13 publics (dont carte, déclaration d'accessibilité, plan du site, 404), 5 participant, 6 organisateur, 5 administrateur. Design system accessible (jetons de couleur vérifiés, police Atkinson Hyperlegible auto-hébergée), préférences d'accessibilité persistées, dialogues accessibles, notifications annoncées, découpage du code par route.

**Différenciant** : carte interactive accessible (Leaflet, clavier, liste équivalente), export iCal, lecture vocale des descriptions.

**DevOps** : Dockerfiles, `docker-compose.yml` (mongo + api + web, volumes, health checks), nginx (proxy, cache, CSP), GitHub Actions (CI complète + publication d'images), Playwright, k6, Lighthouse CI.

## 2. Ce qui a été testé

| Suite | Statut |
|---|---|
| 99 tests unitaires backend | **exécutés — 99 réussis** |
| 6 tests frontend (schémas, formatage) | **exécutés — 6 réussis** |
| 17 tests E2E Playwright (parcours 1 à 9, sécurité, clavier, dialogue, zoom 200 %, mobile) | **exécutés sur le banc de prévisualisation — 17 réussis** |
| Contrôles UI automatisés (structure, alt, labels, noms, contraste, reflow) sur 30 écrans + 12 parcours | **exécutés — 0 anomalie, 12/12** |
| 25 tests d'intégration API + MongoDB | écrits, **non exécutés** |
| 10 tests axe WCAG 2.1 AA | écrits, **non exécutés** |

| Suite | Statut |
| --- | --- |
| Build Docker, CI GitHub, Lighthouse, k6, WAVE, lecteurs d'écran | **non exécutés** |

Le banc de prévisualisation exécute le vrai frontend et la vraie API (routes, contrôleurs, middlewares, validateurs, services) avec des données en mémoire au lieu de MongoDB, parce que l'environnement de développement n'avait accès ni à npm ni à MongoDB ni à Docker. Détails : `TEST-REPORT.md`.

## 3. Résultats réels

- Unitaires backend : `tests 99 · pass 99 · fail 0`.
- E2E (banc) : `13 passed` (parcours + sécurité) et `4 passed` (accessibilité hors axe).
- 6 défauts réels trouvés et corrigés pendant la vérification (voir `TEST-REPORT.md` §5).

## 4. Couverture

Lignes 91,6 % · branches 84,2 % · fonctions 82,9 % sur les 17 fichiers de la couche métier chargés par les tests unitaires. La couche Express/Mongoose (controllers, routes, repositories, modèles) n'est pas incluse : la couverture globale reste à mesurer avec `npm run test:coverage`.

## 5. Accessibilité

Mise en œuvre détaillée par critère dans `deliverables/ACCESSIBILITY-REPORT.md`. Contrôles exécutés : 0 anomalie sur 30 écrans (dont 2 598 nœuds texte au contraste mesuré), clavier, piège de focus des dialogues, récapitulatif d'erreurs, reflow 200 % et mobile. Statut : **partiellement conforme** — axe, Lighthouse, WAVE et tests NVDA/VoiceOver restent à faire.

## 6. Performance

Mesurée sur le banc (réseau limité, CPU ×4) : LCP accueil 8 988 → 2 720 ms, liste 14 220 → 3 840 ms ; JS initial 284 → 87 Ko gzip ; images de la liste −92 % ; CLS liste 0,376 → 0 ; proxy INP ≤ 120 ms. Scores Lighthouse et temps API sous charge k6 **non mesurés**. Détails : `deliverables/PERFORMANCE-REPORT.md`.

## 7. Déploiement

**Non déployé** : aucune URL publique n'existe. `docker compose config` est valide ; la construction et le démarrage des conteneurs n'ont pas pu être exécutés ici. Procédure Render + MongoDB Atlas : `deliverables/DEPLOYMENT.md`.

## 8. Limites connues

1. Tests d'intégration, axe, Docker, CI, Lighthouse, k6 non exécutés (voir §2) : un défaut d'intégration Express/Mongoose reste possible tant qu'ils n'ont pas tourné.
2. Les `package-lock.json` n'existent pas encore (générés au premier `npm install`) ; les versions sont fixées par plages `^` sur des versions majeures stables.

3. Pas de SSR : les fiches événement ne sont pas pré-rendues pour les moteurs de recherche (métadonnées globales seulement).
4. Le formulaire de réservation limite l'affichage à 4 places, même si l'administrateur change le plafond (l'API applique bien le paramètre).
5. Les fonds de carte OpenStreetMap n'ont pas d'alternative textuelle (liste fournie) et dépendent d'un service tiers.
6. Pas de paiement en ligne : les prix sont « à régler sur place » (aucun paiement simulé).
7. En production avec `MAIL_MODE=log`, aucun e-mail n'est envoyé (les liens de réinitialisation apparaissent seulement dans les journaux du conteneur) : configurer un SMTP.
8. Front et API sur deux domaines différents : adapter `SameSite` du cookie (voir `DEPLOYMENT.md`).
9. Les comptes de démonstration ont un mot de passe public : à supprimer avant toute mise en ligne.
10. Les captures fournies proviennent du banc de prévisualisation (React 19, carte sans tuiles).

## 9. Démarrage exact

```
cp .env.example .env                    # renseigner JWT_SECRET
docker compose up --build -d            # http://localhost:8080 · Swagger /api-docs
docker compose down                     # arrêt (données conservées)
```

Sans Docker : `cd backend && cp .env.example .env && npm install && npm run seed && npm run dev`, puis `cd frontend && npm install && npm run dev` (http://localhost:5173). Comptes : admin@ / orga@ / user@inclusivenue.tn, mot de passe `Password123`.

## 10. Checklist de remise

Voir `deliverables/CHECKLIST.md` (état de chaque livrable) et `deliverables/A-FOURNIR.md` (ce que l'étudiant doit encore produire : URL du dépôt, URL déployée, preuves d'exécution, captures réelles, audits).